

Auditoria Técnica Apurada do PLANOS_LUAN

Versão refinada a partir da conferência do código, do banco e do relatório anterior

Data da apuração: 02/07/2026

Escopo desta versão

Esta versão apurada foi montada a partir de quatro bases reais:

- leitura do PDF Auditoria Planos Luan.pdf;
- conferência do banco planos_luan.db;
- leitura pontual de config.py, core/database.py, core/revisao_final.py e planos_luan_app.py;
- conferência visual do próprio PDF para separar problema de conteúdo de problema de apresentação.

O objetivo aqui não é reescrever o relatório anterior do zero. É preservar o que ele acertou, corrigir o que ficou impreciso e transformar o diagnóstico em um plano mais seguro para o sistema real.

1. Parecer executivo

O relatório anterior acertou o ponto principal: o maior problema do PLANOS_LUAN não é "código ruim" isoladamente. O problema estrutural é a ausência de fronteiras claras entre:

- código-fonte;
- dados operacionais;
- histórico de geração;
- laboratório manual;
- contratos de dados do próprio fluxo.

Também está correto dizer que os arquivos centrais cresceram demais e concentram responsabilidades demais, especialmente core/lote.py e planos_luan_app.py.

Onde a versão anterior exagerou ou simplificou além do ponto foi em três lugares:

- historico_planos.arquivo_path não guarda hoje um caminho absoluto completo do Windows. O fluxo atual salva o arquivo em HISTORICO_DOCX_DIR e grava no banco apenas unique_filename.
- A revisão pós-geração foi desabilitada na interface, mas a camada de auditoria não sumiu do código. confidence_score e avisos_validacao continuam sendo calculados e gravados no sidecar.
- Algumas propostas de modularização estão bem direcionadas, mas ainda são hipóteses arquiteturais. Elas precisam ser guiadas por teste e extração gradual, não por corte cego.

Conclusão executiva: o diagnóstico estrutural é bom e merece ser aproveitado. Mas a execução deve partir da versão apurada abaixo, não da leitura literal do PDF anterior.

2. O que o relatório anterior acertou

Ponto	Leitura	Avaliação
Falta de fronteiras	O projeto mistura sistema, operação e laboratório na mesma árvore	Correto
Arquivos centrais extensos	core/lote.py com 3915 linhas e planos_luan_app.py com 3495 linhas	Correto
Contrato de dados parcial	O modelo formal da IA cobre menos do que o fluxo realmente manipula	Correto
Acoplamento alto	UI, regra de negócio, persistência e operação de arquivos estão muito próximas	Correto
Risco de regressão silenciosa	O sistema tem vários pontos onde mudanças pequenas podem produzir efeito lateral	Correto

3. O que precisava de ajuste

Item	Relatório anterior	Versão apurada	Efeito prático
arquivo_path no banco	Tratado como caminho absoluto completo do DOCX	Hoje o banco grava o nome relativo unique_filename, resolvido depois com HISTORICO_DOCX_DIR	O risco continua existindo, mas é um acoplamento entre banco e pasta-base, não um caminho absoluto salvo linha a linha
Revisão pós-geração	Tratada como inexistente	A revisão visual da interface foi desligada, mas a auditoria técnica ainda calcula confidence_score e avisos_validacao	O problema real é que a validação perdeu função operacional direta, não que ela tenha sido apagada
Separação proposta de módulos	Apresentada em alguns trechos como se já estivesse provada	Parte dela é bem fundamentada; parte ainda depende de extração assistida por testes	Evita refatoração por impulso

4. Fotografia estrutural apurada

No estado atual da pasta de trabalho, o sistema convive com:

- 194 arquivos .py;
- 527 arquivos .docx;
- 55 arquivos .pdf;
- quase 1000 arquivos .png;
- 18 professores cadastrados;
- 147 vínculos em professor_turmas;
- 452 registros em historico_planos.

Isso confirma que a árvore do projeto não está guardando apenas software. Ela também está sendo usada como área operacional, histórico e laboratório.

5. Evidências concretas conferidas

Evidência	O que foi observado	Leitura apurada
config.py	DB_PATH = BASE_DIR / "planos_luan.db"	O banco principal continua dentro da raiz do projeto
config.py	HISTORICO_DOCX_DIR = BASE_DIR / "historico_docx"	O histórico físico dos DOCX também continua na raiz
config.py	HABILITAR_REVISAO_POS_GERACAO = False	A revisão pós-geração foi desligada na interface
core/database.py	O arquivo é salvo em Path(HISTORICO_DOCX_DIR) / unique_filename	O banco depende da pasta-base do histórico para reencontrar o DOCX
core/database.py	O insert grava arquivo_path = unique_filename	O valor persistido hoje é relativo ao diretório do histórico, não um caminho absoluto completo
core/revisao_final.py	revisar_aula_gerada() calcula confidence_score e avisos_validacao	A camada de auditoria técnica continua viva
core/revisao_final.py	gravar_sidecar_json() salva metadados de auditoria no JSON	O sistema ainda produz rastros de validação
planos_luan_app.py	Quando a flag de revisão está falsa, o estado de revisão é limpo	A interface deixou de usar a revisão como etapa obrigatória

6. Diagnóstico apurado por prioridade

Crítico

Risco	Por que é crítico	Ajuste de leitura
-------	-------------------	-------------------

Dados operacionais dentro da pasta do projeto	Banco, histórico e laboratório dividem espaço com o código	Este é o risco mais imediato e mais fácil de reduzir sem reescrever o sistema
Ausência de backup automático do banco	Perda ou corrupção do SQLite afeta histórico real de geração	A criticidade aqui é operacional, não apenas arquitetural
Contrato de dados incompleto no fluxo do plano	O objeto real circulante tem muitos campos fora do modelo principal	Isso alimenta deriva silenciosa e dificulta qualquer refatoração

Alto

Risco	Por que é alto	Ajuste de leitura
core/lote.py como centro de orquestração	Um arquivo conhece regras demais e conversa com módulos demais	O risco não é tamanho por si; é concentração de decisão
planos_luan_app.py misturando UI com regra	Dificulta teste, rastreo e manutenção	A interface está assumindo responsabilidades além da apresentação
Validação sem uso operacional forte	Os avisos ainda existem, mas não estão governando a decisão final	O problema é de calibração e encaixe no fluxo

Moderado

Risco	Por que é moderado	Ajuste de leitura
Caminhos muito orientados a Windows e pasta local	O sistema depende de convenções específicas de máquina	É um limitador de portabilidade e teste
EXCLUIR como área de laboratório permanente	Pode gerar ruído humano e técnico	O risco é real, mas fácil de atacar com higiene operacional
Campos livres em tabelas simples	Valores inconsistentes podem escapar	Problema relevante, mas não é a primeira frente

7. O que eu faria primeiro, na prática

Etapa 0 - Higiene operacional e segurança mínima

Prazo sugerido: 1 a 2 dias

- Criar uma pasta de dados fora do código, como D:\PLANOS_LUAN_DATA.
- Planejar a mudança de planos_luan.db, historico_docx, historico e EXCLUIR para fora da árvore principal.
- Ajustar config.py para apontar para essas novas bases.

- Criar backup manual antes e depois da mudança.
- Testar uma geração completa logo em seguida.

Resultado esperado: a pasta do projeto volta a parecer um projeto de software, e não um depósito misto de código e operação.

Etapa 1 - Contrato único do plano

Prazo sugerido: 3 a 5 dias

- Criar `core/models.py`.
- Definir `PlanoCompleto` cobrindo os campos usados no fluxo e no sidecar.
- Encapsular `PlanoAulaIA` dentro desse contrato maior, sem descartar o modelo atual.
- Fazer `gravar_sidecar_json()` serializar o contrato tipado.

Resultado esperado: qualquer função importante do pipeline passa a trabalhar com um objeto conhecido, e não com dicionários que crescem no meio do caminho.

Etapa 2 - Reaproveitar a validação sem recolocar ruído na interface

Prazo sugerido: 1 a 2 dias

- Manter `HABILITAR_REVISAO_POS_GERACAO = False` por enquanto.
- Criar um script offline, como `scripts/validar_sidecars.py`.
- Rodar os sidecars já gerados para entender padrões reais de aviso.
- Só depois decidir se a revisão volta para a interface, e em que formato.

Resultado esperado: a validação volta a ser útil sem travar a operação diária.

Etapa 3 - Quebrar `core/lote.py` com contrato já definido

Prazo sugerido: 1 a 2 semanas

- Extrair primeiro o executor de um único plano.
- Separar depois o seletor de referências.
- Deixar `lote.py` original como fachada temporária durante a transição.

Resultado esperado: o sistema começa a ganhar pontos testáveis sem uma ruptura brusca.

Etapa 4 - Só então mexer na fronteira UI x regra

Prazo sugerido: 1 a 2 semanas

- Tirar de `planos_luan_app.py` tudo o que não depende de `st.*`.
- Levar essa lógica para um módulo de operação no `core`.
- Deixar a UI como camada de entrada, exibição e estado.

Resultado esperado: a interface deixa de ser o lugar onde o comportamento real mora.

8. O que eu não faria agora

- Não reativaria a revisão pós-geração na interface sem calibrar os avisos offline.
- Não refatoraria `core/lote.py` e `planos_luan_app.py` ao mesmo tempo.
- Não mexeria em `docx_generator/preencher.py` antes de ter teste de regressão do documento gerado.
- Não faria migração de schema do SQLite como primeira frente.
- Não trataria a proposta de nova arquitetura como "corte estrutural único". O caminho aqui precisa ser incremental.

9. Decisão recomendada para os próximos 7 dias

Ordem	Ação	Saída concreta
1	Definir a nova pasta de dados fora do projeto	Caminhos separados para banco, histórico e laboratório
2	Ajustar <code>config.py</code> e validar inicialização	Sistema acusando caminho quebrado logo no início
3	Criar rotina simples de backup do SQLite	Segurança mínima operacional
4	Criar <code>core/models.py</code> com <code>PlanoCompleto</code>	Contrato único para o fluxo
5	Criar um script de validação offline dos sidecars	Reaproveitamento útil da camada de auditoria

10. Conclusão apurada

O relatório anterior é bom como disparador de conversa, porque acerta o centro do problema: o PLANOS_LUAN cresceu sem fronteiras suficientemente claras.

Mas a leitura mais fiel do sistema hoje é esta:

- O risco mais urgente é operacional: dados reais morando dentro da pasta do projeto.
- O risco mais corrosivo é lógico: um contrato de dados incompleto para o plano final.
- O risco mais caro de manutenção é técnico: poucos arquivos tomando decisões demais.

O sistema não precisa ser jogado fora. Ele precisa ser reencaixado por etapas, com sequência, teste e menos improviso estrutural. A ordem importa mais do que a velocidade.

Se a equipe seguir essa sequência, há caminho real para estabilizar o projeto sem interromper o uso do sistema no dia a dia.